AC-SINDy: Compositional Sparse Identification of Nonlinear Dynamics

Peter Racioppo
Independent Researcher
Los Angeles, CA 90731
pcracioppo@gmail.com

March 2023 (updated March 2026)

Abstract

We present AC-SINDy, a compositional extension of the Sparse Identification of Nonlinear Dynamics (SINDy) framework that replaces explicit feature libraries with a structured representation based on arithmetic circuits. Rather than enumerating candidate basis functions, the proposed approach constructs nonlinear features through compositions of linear functions and multiplicative interactions, yielding a compact and scalable parameterization and enabling sparsity to be enforced directly over the computational graph. We further introduce a formulation that separates state estimation from dynamics identification by combining latent state inference with shared dynamics and multi-step supervision, improving robustness to noise while preserving interpretability. Experiments on nonlinear and chaotic systems demonstrate that the method recovers accurate and interpretable governing equations while scaling more favorably than standard SINDy.

1 Introduction

Discovering governing equations of dynamical systems from data is a central problem in scientific machine learning. The Sparse Identification of Nonlinear Dynamics (SINDy) framework addresses this by selecting a sparse combination of candidate basis functions from a predefined library [Brunton et al., 2016], enabling recovery of compact and interpretable models.

However, SINDy suffers from a fundamental scalability limitation: the size of the candidate library grows combinatorially with the state dimension and the desired degree of nonlinearity. Representing higher-order or compositional dynamics therefore requires explicit enumeration of many candidate terms, leading to increased computational cost and sensitivity to feature design. Moreover, SINDy relies on accurate derivative estimation and is sensitive to noise, limiting its applicability in realistic settings.

In this work, we reformulate SINDy by replacing explicit feature libraries with a compositional, learnable representation. Rather than expressing the dynamics as a linear combination of predefined functions, we construct them implicitly through a structured architecture that composes simple primitives such as linear transformations and multiplicative interactions. This yields a compact, factorized parameterization that can be interpreted as a sparse arithmetic circuit, providing a scalable alternative to explicit basis expansion.

This reformulation induces a shift in the underlying hypothesis class. Standard SINDy represents all candidate functions up to a given complexity, whereas the proposed approach represents a structured subset defined by compositional factorization. While this sacrifices explicit completeness, it enables efficient representation of individual interactions under sparsity assumptions, and avoids the combinatorial growth of feature libraries.

Preprint.

Beyond representation, we address the challenge of learning dynamics from noisy observations. Models trained on one-step prediction can implicitly combine filtering and dynamics, leading to solutions that fit data but do not correspond to a consistent underlying system. To address this, we introduce a formulation that separates state estimation from dynamics identification: a latent state is inferred from observations and evolved using a shared dynamics model trained with multi-step supervision. This enforces consistency under repeated application and enables robust learning while preserving interpretability.

We further introduce *Feature Normalization*, a scale-invariant parameterization that ensures learned coefficients reflect functional importance rather than feature magnitude. This is particularly important in compositional models, where intermediate features can vary significantly in scale.

Finally, we enforce sparsity through iterative pruning using gradient-based importance estimates, enabling structure learning in the shared computational graph.

We evaluate the method on a range of nonlinear and chaotic dynamical systems, including settings with noisy observations. Results demonstrate that the approach recovers accurate and interpretable models while scaling more favorably than standard SINDy.

Our contributions are as follows:

- We introduce AC-SINDy, a compositional reformulation of SINDy based on arithmetic circuits, replacing explicit feature libraries with a structured, factorized representation.
- We propose a formulation that separates filtering from dynamics identification via latent state estimation and multi-step consistency, enabling robust learning from noisy data.
- We introduce Feature Normalization, a scale-invariant parameterization that stabilizes optimization and improves interpretability in compositional models.
- We develop a pruning-based structure learning procedure for shared computational graphs and demonstrate recovery of sparse, interpretable dynamics across nonlinear and chaotic systems.

2 Related Work

The discovery of governing equations from data has been significantly advanced by the Sparse Identification of Nonlinear Dynamics (SINDy) framework [Brunton et al., 2016], which employs sparse regression over a fixed library of candidate functions to obtain interpretable models. However, this approach is sensitive to noise, depends on the choice of coordinates, and suffers from combinatorial growth in library size.

Several extensions have addressed these limitations. Rudy et al. [2016] improved robustness to noise through regularization and numerical differentiation, while Kaheman et al. [2020] introduced SINDy-PI to handle implicit and rational dynamics. Champion et al. [2019] proposed the SINDy Autoencoder, which learns a latent coordinate system in which the dynamics are sparse, demonstrating that neural networks can aid in discovering parsimonious representations.

A central challenge in SINDy is estimating time derivatives from noisy data. Both et al. [2021] introduced DeePyMoD, which uses automatic differentiation within neural networks to estimate derivatives directly. Forootani et al. [2023] further improve robustness by replacing differential formulations with integral constraints. Related work has also combined SINDy with Neural Ordinary Differential Equations (Neural ODEs), enabling multi-step training and the enforcement of physical structure through backpropagation in time [Lee et al., 2021].

More recent work has begun to relax the fixed-library assumption by learning or adapting candidate functions directly from data. These approaches include parameterized or learned libraries [Yonezawa et al., 2026, Singh and Mukherjee, 2024], as well as architectural extensions such as nested or hierarchical SINDy formulations [Fiorini et al., 2025]. In parallel, advances in neural symbolic regression and compositional architectures, such as Kolmogorov–Arnold Networks, highlight the potential of learned functional representations [Faroughi et al., 2025].

Despite these advances, most existing approaches retain an explicit library of candidate functions or a parametric variant thereof. As a result, they continue to rely on basis expansion, with associated scalability challenges as the state dimension and desired model complexity increase.

In contrast, our approach replaces explicit libraries with a compositional representation based on arithmetic circuits. Rather than enumerating candidate functions, the model constructs features through compositions of linear transformations and multiplicative interactions, defining an implicit hypothesis class. Sparsity is enforced over the structure of this computation, yielding a structured, factorized representation of the dynamics. This formulation trades explicit completeness of the function library for improved parameter efficiency and scalability.

3 Background: The SINDy Algorithm

Consider a first-order nonlinear dynamical system of the form

$$\dot{x}(t) = f(x(t), u(t), t),$$

where $x(t) \in \mathbb{R}^n$ is the state vector and $u(t)$ denotes control inputs. The Sparse Identification of Nonlinear Dynamics (SINDy) framework seeks to approximate the unknown function f using a sparse linear combination of candidate basis functions.

Specifically, for each component $x_i(t)$, $i \in \{1, \ldots, n\}$, SINDy solves a regression problem of the form

$$\mathcal{L}_i = \|\dot{x}_i(t) - g_i(x(t), t)\|_2^2,$$

where

$$g_i(x(t), t) = \sum_j w_{i,j} f_{i,j}(x(t), t)$$

is a linear combination of predefined basis functions $\{f_{i,j}\}$. The key assumption is that only a small subset of these basis functions is required to accurately describe the system dynamics. The SINDy algorithm is illustrated in Fig. 1.

In practice, SINDy performs least-squares regression combined with iterative thresholding, sequentially removing coefficients with small magnitude (typically based on an L1 criterion). Empirically, this procedure has been observed to outperform standard LASSO-based approaches in identifying sparse and interpretable models.

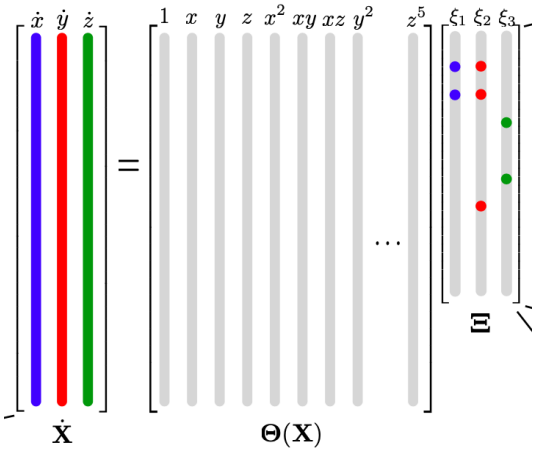

Figure 1: Illustration of SINDy regression, reproduced from the original work.

4 AC-SINDy: Compositional Sparse Identification via Arithmetic Circuits

We propose a compositional extension of the SINDy framework in which the explicit library of candidate functions is replaced by a structured neural architecture. The model constructs nonlinear features through compositions of learned transformations, enabling a compact and scalable representation of dynamical systems.

While the architecture can be interpreted as a sparse arithmetic circuit when restricted to linear and multiplicative operations, this is a special case of a more general compositional framework.

Additional nonlinear primitives (e.g., sinusoidal or other parametric functions) can be incorporated, yielding a broader class of neural SINDy models.

4.1 Architecture

Let $x \in \mathbb{R}^n$ denote the system state. At each layer ℓ, we form an augmented input

$$\tilde{x}^{(\ell)} = [x^{(\ell)}; 1] \in \mathbb{R}^{n_\ell+1},$$

where the constant feature enables representation of bias and lower-order terms.

Each layer consists of:

Masked Linear layer.

$$h^{(\ell)} = W^{(\ell)}\tilde{x}^{(\ell)}, \quad W^{(\ell)} \in \mathbb{R}^{d_\ell \times (n_\ell+1)}.$$

Interaction layer. Features are combined via nonlinear interactions. In the simplest case:

$$x_k^{(\ell+1)} = \prod_{j \in \mathcal{G}_k} h_j^{(\ell)},$$

where $\{\mathcal{G}_k\}$ partitions the features. More generally, other nonlinear primitives may be used.

Stacking layers yields a compositional computation graph in which features are built recursively. The compositional structure admits automatic symbolic extraction via propagation of polynomial expressions through the network.

4.2 Function Class

The model represents functions of the form:

$$f(x) = \sum_{k=1}^{m} \psi_k(x),$$

where each ψ_k is constructed by the network. In the multiplicative case:

$$\psi_k(x) = \prod_{j \in S_k} \left(w_{k,j}^T x\right).$$

Unlike standard SINDy, which uses a fixed library $\Theta(x)$, the model defines an implicit hypothesis class through composition and does not enumerate all candidate functions.

4.3 Expressivity and Sparse Compositional Structure

The architecture does not guarantee representation of all polynomial terms simultaneously. For example, a quadratic function

$$f(x) = x^T Q x$$

requires independent coefficients for each pair (i, j), whereas the model represents such functions as

$$f(x) = \sum_{k=1}^{r} (a_k^T x)(b_k^T x),$$

corresponding to a low-rank factorization of Q.

More generally, the model need not span the full function space, but can represent individual interactions or small subsets thereof. This leads to a different assumption: the dynamics admit a sparse compositional representation. Under this assumption, the model can recover the true dynamics by selecting a small number of active terms.

4.4 Interpretation as an Arithmetic Circuit

When restricted to linear and multiplicative operations, the model can be viewed as a sparse arithmetic circuit:

- Linear layers correspond to addition (sum nodes)
- Multiplicative layers correspond to product nodes
- Pruning removes edges in the computational graph

This highlights the role of compositional reuse: intermediate features can be shared across multiple terms.

4.5 Sparsification

Sparsity is enforced by pruning weights in the linear layers. Each parameter corresponds to an edge in the computational graph, and pruning removes edges with minimal impact on performance. This yields a sparse structure in which only a subset of interactions is active.

Unlike standard SINDy, sparsity operates over both feature construction and composition.

4.6 Parameter Efficiency

In standard SINDy, polynomial libraries lead to combinatorial growth. The number of monomials of degree p in n variables is

$$\binom{n+p-1}{p},$$

which counts the number of ways to distribute degree across variables. Including all terms up to order p:

$$\sum_{k=0}^{p} \binom{n+k-1}{k} = \frac{1}{n}(p+1)\binom{n+p}{p+1} \sim \mathcal{O}(pn^p).$$

In contrast, each compositional term requires $\mathcal{O}(pn)$ parameters. If m terms are active, the total parameter count is $\mathcal{O}(mpn)$, yielding $\mathcal{O}(n^2p)$ under $m = \mathcal{O}(n)$.

This improved scaling arises from restricting the hypothesis class to factorized, compositional representations rather than enumerating all monomials. While the model does not represent all terms simultaneously, it is sufficient when the underlying dynamics are sparse.

Figure 2 illustrates this difference in scaling.

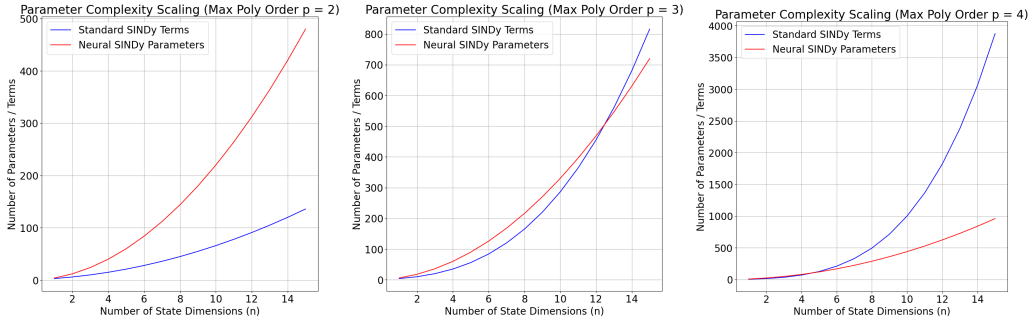

Figure 2: Parameter growth for standard SINDy (explicit enumeration) versus AC-SINDy (compositional representation) for polynomial orders $p = 2, 3, 4$.

5 Importance-Based Pruning in Compositional Models

SINDy enforces sparsity by selecting a small number of active terms from a predefined library. In this setting, each parameter corresponds to a single basis function, and coefficient magnitude provides a reasonable proxy for importance.

In contrast, the proposed model represents functions through a shared computational graph. Parameters correspond to edges that may contribute to many compositional terms, breaking the one-to-one correspondence between parameters and functions. As a result, magnitude-based sparsification is no longer well-aligned with functional importance.

To address this, we consider the marginal contribution of each parameter,

$$\Delta_i = \mathcal{L}(w_{-i}) - \mathcal{L}(w),$$

which measures the effect of removing a parameter on the loss. While this provides a principled notion of importance, computing it exactly is not scalable.

Instead, we approximate this quantity using a first-order Taylor expansion,

$$\Delta_i \approx \left| w_i \frac{\partial \mathcal{L}}{\partial w_i} \right|,$$

yielding a gradient-based pruning criterion. This approximation captures both the magnitude of a parameter and its influence on the loss, while remaining computationally efficient.

In practice, we track a moving average of the training loss and trigger pruning once optimization plateaus. At each pruning step, a subset of parameters with the lowest estimated importance is removed, after which the model is finetuned to recover performance. Repeating this process produces a sequence of progressively sparser models, from which the final model is selected based on validation loss.

6 Feature Normalization

A key challenge in both SINDy and compositional models is sensitivity to feature scaling. Candidate functions such as x, x^2, and higher-order terms can differ significantly in magnitude depending on the region of the state space. As a result, learned coefficients depend inversely on feature scale, making coefficient magnitude an unreliable indicator of functional importance and distorting sparsity-based model selection.

To address this, we introduce *Feature Normalization*, which rescales each feature independently using statistics computed over the data. Let $h(x) \in \mathbb{R}^d$ denote the feature vector produced by the model. We define

$$\tilde{h}_i(x) = \frac{h_i(x)}{\sigma_i + \epsilon},$$

where σ_i is the standard deviation of feature h_i estimated over the training trajectory. The model output is then

$$f(x) = W\tilde{h}(x).$$

In contrast to standard normalization methods, Feature Normalization preserves identifiability. Methods such as Batch Normalization and Layer Normalization allow features to be arbitrarily rescaled through learnable affine parameters and gradient-dependent normalization statistics. As a result, the relationship between coefficient magnitude and functional importance is not preserved. Feature Normalization avoids this by operating independently on each feature and treating normalization statistics as constants (via stop-gradient), ensuring a stable, scale-invariant, and interpretable parameterization.

To avoid introducing instability, we compute σ_i using running statistics but detach it from the computational graph:

$$\tilde{h}_i(x) = \frac{h_i(x)}{\text{stopgrad}(\sigma_i) + \epsilon}.$$

This yields a scale-invariant formulation for sparse system identification in compositional models.

7 Multi-Step Consistency and Separation of Filtering and Dynamics

A central challenge in learning dynamical systems from data is disentangling state estimation from dynamics identification. In many settings, observations are noisy or partial, and models trained on

one-step prediction can implicitly combine filtering and dynamics into a single function. While such models may fit the data, they are not constrained to represent a time-consistent dynamical system.

To address this, we adopt a formulation in which a single dynamics function is applied repeatedly over time. Let S_t denote a latent state estimate and f_θ a learned dynamics model. We define the evolution

$$S_{t+1} = S_t + f_\theta(S_t)\,\Delta t, \qquad S_{t+k} = \Phi_\theta^{(k)}(S_t),$$

where $\Phi_\theta^{(k)}$ denotes k successive applications of f_θ. This shared-parameter structure enforces consistency under composition.

Crucially, rollout is combined with multi-step supervision,

$$\mathcal{L} = \sum_{k=1}^{K} \|\hat{x}_{t+k} - x_{t+k}\|^2 ,$$

which enforces accuracy over extended horizons. Without this constraint, an expressive encoder can encode future information into S_t, allowing trivial dynamics to minimize one-step loss. Multi-step supervision eliminates this degeneracy by requiring consistent predictions across multiple time steps.

Under this formulation, the model decomposes into a filtering component and a dynamics component:

$$S_t = \mathcal{E}(x_{1:t}), \qquad S_{t+k} = \Phi_\theta^{(k)}(S_t).$$

The encoder $\mathcal{E}$ estimates the latent state from observations, while f_θ governs its evolution. Because f_θ is applied repeatedly and evaluated over multiple steps, it must represent a self-consistent dynamical law rather than relying on access to the observation history.

This structure aligns with classical system identification, where a state estimate is propagated using a fixed model. In contrast to one-step regression, the multi-step formulation enforces temporal consistency and discourages solutions that do not generalize under rollout.

8 Control with Structured Dynamics

The proposed framework naturally supports model-based control by providing an explicit, interpretable representation of the system dynamics. Given a learned model

$$\dot{x} = g_w(x, u),$$

standard control techniques can be applied, including linearization and trajectory optimization.

A key advantage of AC-SINDy over black-box neural models is the structured and sparse form of g_w. This enables direct reasoning about the dynamics and, in some cases, closed-form controller design. For example, consider a system of the form

$$\dot{x} = p_1(x) + ku,$$

where $p_1(x)$ is approximated by a sparse polynomial. If the desired dynamics are $\dot{x} = p_2(x)$, and the control input is parameterized as a polynomial $u = q_\theta(x)$, then the closed-loop system remains polynomial, and the controller parameters can be chosen to match coefficients:

$$\theta_i \approx \frac{b_i - a_i}{k}.$$

More generally, the compositional structure of the learned model allows for efficient local linearization and application of classical control methods such as LQR. Compared to dense neural representations, the sparse and structured form of AC-SINDy models may lead to more stable and interpretable control policies.

We leave a detailed empirical investigation of control applications to future work.

9 Experimental Results

We implemented the proposed AC-SINDy architecture in PyTorch.[1]

[1] `https://github.com/PCR-git/AC-SINDy`

9.1 Experimental Setup

We evaluate the method on low-dimensional dynamical systems, including canonical chaotic and nonlinear systems.

2D nonlinear system.

$$\dot{x}(t) = -0.1x + y, \quad \dot{y}(t) = -2x - 0.1y - 0.5xy - 0.025y^2.$$

Lorenz system.

$$\dot{x} = \sigma(y - x), \quad \dot{y} = x(\rho - z) - y, \quad \dot{z} = xy - \beta z,$$

with $(\sigma, \rho, \beta) = (10, 28, 8/3)$.

Lorenz system with sinusoidal forcing.

$$\dot{x}(t) = \sigma(y - x) + 0.1\sin(x), \quad \dot{y}(t) = x(\rho - z) - y, \quad \dot{z}(t) = xy - \beta z.$$

9.2 Training Dynamics

Figure 3 shows the training loss during iterative pruning. Each pruning step induces a temporary increase in loss, followed by recovery during retraining. For systems with sparse underlying dynamics, pruning initially improves performance by removing redundant parameters, but eventually leads to underfitting. The final model is selected at the minimum validation loss, corresponding to the optimal sparsity–expressivity tradeoff.

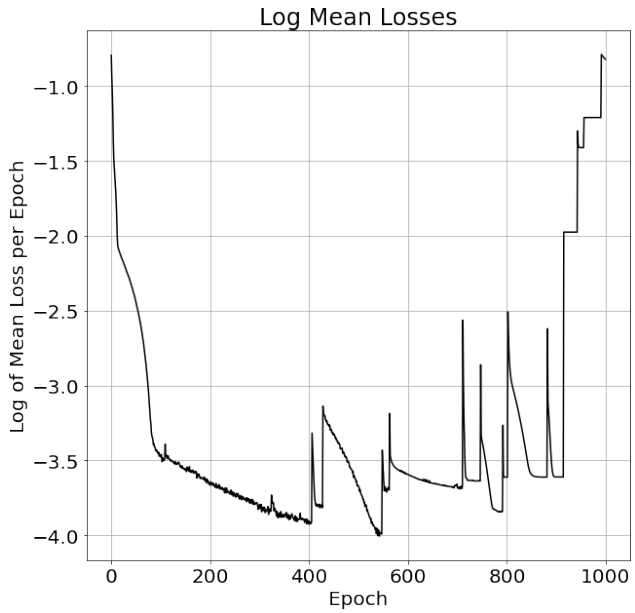

Figure 3: Training loss during iterative pruning. Loss spikes occur when parameters are removed, followed by recovery as the model re-optimizes. Excessive pruning degrades performance.

9.3 Prediction Quality

Figure 4 shows predictions on the Lorenz system. The model accurately captures both individual state dynamics and the global phase-space structure. As expected for chaotic systems, long-term trajectories diverge from the ground truth.

9.4 Model Recovery

A primary objective of SINDy-based methods is recovery of the underlying governing equations. The proposed approach identifies sparse and interpretable structure, including nonlinear interaction terms.

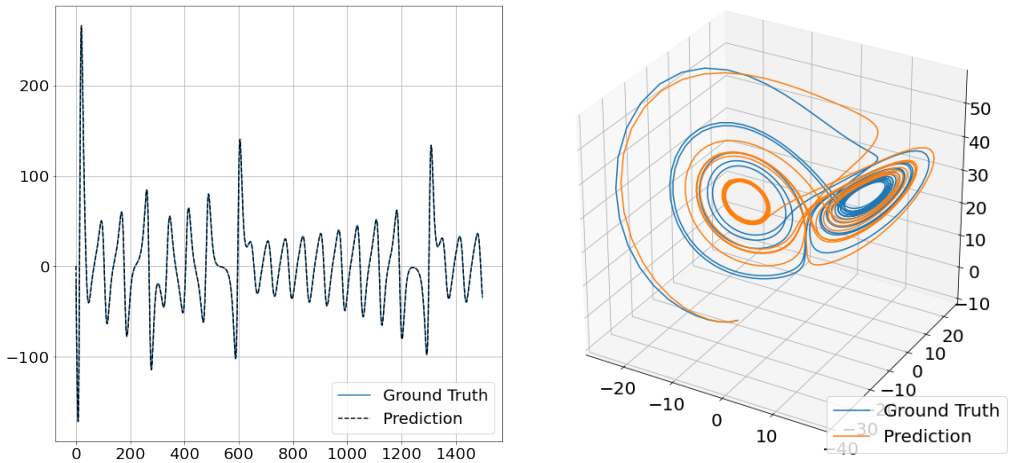

Figure 4: Left: Predicted vs. true trajectory for a single state variable. Right: Phase-space trajectory. The learned dynamics capture the qualitative structure despite long-term divergence.

2D nonlinear system.

$$\text{True dynamics:} \quad \dot{x} = -0.1x + y, \quad \dot{y} = -2x - 0.1y - 0.5xy - 0.025y^2.$$

$$\text{Recovered system:} \quad \dot{x} \approx -0.11x + 1.00y, \quad \dot{y} \approx -1.93x - 0.11y - 0.50xy - 0.029y^2.$$

Lorenz system.

$$\text{True dynamics:} \quad \dot{x} = 10(y - x), \quad \dot{y} = x(28 - z) - y, \quad \dot{z} = xy - \tfrac{8}{3}z.$$

$$\text{Recovered system:} \quad \begin{aligned} &\dot{x} \approx 8.72\,y - 7.17\,x, \quad \dot{y} \approx 26.18\,x + 0.90\,y, \\ &\dot{z} \approx 0.25\,x^2 + 0.49\,xy + 0.10\,x + 0.25\,y^2 - 0.03\,y - 7.18. \end{aligned}$$

The learned 2D nonlinear model recovers the correct functional structure. For the Lorenz system, while the exact coefficients are not perfectly recovered and some spurious terms appear, the dominant interactions and qualitative dynamics are correctly identified.

These results demonstrate that the proposed approach can recover accurate dynamical models despite not explicitly enumerating candidate terms, demonstrating that compositional parameterization is sufficient for sparse system identification.

9.5 Noisy Data and Filtering

We further evaluate the proposed approach in a noisy setting using the filtering-based formulation described in Section 7. We consider the same 2D nonlinear system:

$$\dot{x} = -0.1x + y, \quad \dot{y} = -2x - 0.1y - 0.5xy - 0.025y^2,$$

and add Gaussian noise to the observed trajectories:

$$\tilde{x}(t) = x(t) + \epsilon(t), \quad \epsilon \sim \mathcal{N}(0, 0.05^2).$$

Figure 5 shows the noisy observations, ground truth trajectories, and model predictions. Despite significant observation noise, the model is able to fit the trajectories and recover a smooth approximation of the underlying dynamics.

The recovered governing equations are:

$$\text{Recovered system:} \quad \dot{x} \approx -0.08\,x + 0.55\,y, \quad \dot{y} \approx 0.04\,x^2 - 0.57\,xy - 2.14\,x.$$

The recovered model closely matches the dominant linear and bilinear terms, while introducing a small spurious quadratic term (x^2), omitting the weaker y^2 interaction, and exhibiting minor coefficient deviations.

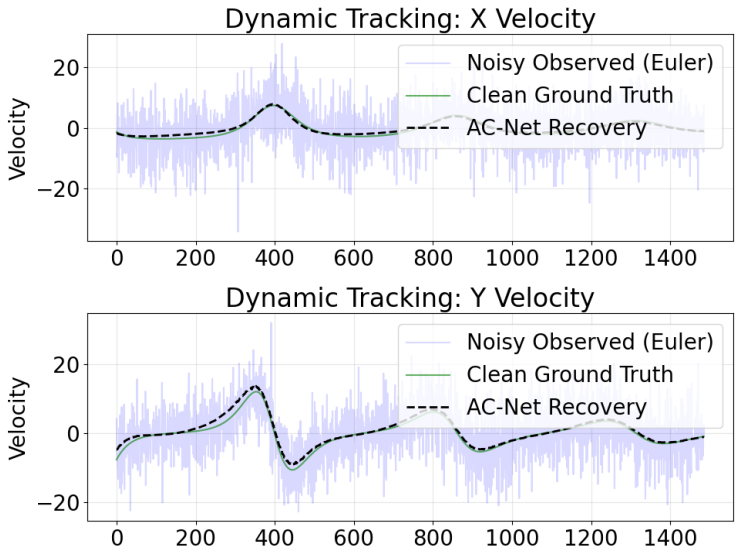

Figure 5: Noisy system identification with Gaussian noise ($\sigma = 0.05$). The model fits the noisy observations while recovering a smooth approximation of the underlying dynamics.

10 Conclusion

We introduced AC-SINDy, a compositional reformulation of SINDy that replaces explicit feature libraries with a structured, factorized representation of nonlinear dynamics. By constructing features through compositions of simple primitives and enforcing sparsity over the resulting computational graph, the method enables interpretable model discovery without explicit basis enumeration. Experiments demonstrate that the approach can recover accurate and compact governing equations.

References

Gert-Jan Both, Subham Choudhury, Pierre Sens, and Remy Kusters. Deepmod: Deep learning for model discovery in noisy data. *Journal of Computational Physics*, 428:109985, March 2021. ISSN 0021-9991. doi: 10.1016/j.jcp.2020.109985. URL `http://dx.doi.org/10.1016/j.jcp.2020.109985`.

Steven L. Brunton, Joshua L. Proctor, and J. Nathan Kutz. Discovering governing equations from data by sparse identification of nonlinear dynamical systems. *Proceedings of the National Academy of Sciences*, 113(15):3932–3937, March 2016. ISSN 1091-6490. doi: 10.1073/pnas.1517384113. URL `http://dx.doi.org/10.1073/pnas.1517384113`.

Kathleen Champion, Bethany Lusch, J. Nathan Kutz, and Steven L. Brunton. Data-driven discovery of coordinates and governing equations. *Proceedings of the National Academy of Sciences*, 116 (45):22445–22451, October 2019. ISSN 1091-6490. doi: 10.1073/pnas.1906995116. URL `http://dx.doi.org/10.1073/pnas.1906995116`.

Salah A. Faroughi, Farinaz Mostajeran, Amin Hamed Mashhadzadeh, and Shirko Faroughi. Scientific machine learning with kolmogorov-arnold networks, 2025. URL `https://arxiv.org/abs/2507.22959`.

Camilla Fiorini, Clément Flint, Louis Fostier, Emmanuel Franck, Reyhaneh Hashemi, Victor Michel-Dansac, and Wassim Tenachi. Generalizing the sindy approach with nested neural networks. *ESAIM: Proceedings and Surveys*, 81:168–192, 2025. ISSN 2267-3059. doi: 10.1051/proc/202581168. URL `http://dx.doi.org/10.1051/proc/202581168`.

Ali Forootani, Pawan Goyal, and Peter Benner. A robust sindy approach by combining neural networks and an integral form, 2023. URL `https://arxiv.org/abs/2309.07193`.

Kadierdan Kaheman, J. Nathan Kutz, and Steven L. Brunton. Sindy-pi: a robust algorithm for parallel implicit sparse identification of nonlinear dynamics. *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 476(2242), October 2020. ISSN 1471-2946. doi: 10.1098/rspa.2020.0279. URL http://dx.doi.org/10.1098/rspa.2020.0279.

Kookjin Lee, Nathaniel Trask, and Panos Stinis. Structure-preserving sparse identification of nonlinear dynamics for data-driven modeling, 2021. URL https://arxiv.org/abs/2109.05364.

Samuel H. Rudy, Steven L. Brunton, Joshua L. Proctor, and J. Nathan Kutz. Data-driven discovery of partial differential equations, 2016. URL https://arxiv.org/abs/1609.06401.

Arunabh Singh and Joyjit Mukherjee. Learn: Learnable and adaptive representations for nonlinear dynamics in system identification, 2024. URL https://arxiv.org/abs/2412.12036.

Ansei Yonezawa, Heisei Yonezawa, Shuichi Yahagi, Itsuro Kajiwara, Shinya Kijimoto, Hikaru Taniuchi, and Kentaro Murakami. Sparse identification of nonlinear dynamics with library optimization mechanism: Recursive long-term prediction perspective. *IEEE Transactions on Cybernetics*, page 1–14, 2026. ISSN 2168-2275. doi: 10.1109/tcyb.2026.3652850. URL http://dx.doi.org/10.1109/TCYB.2026.3652850.